



| WebSockets

В современных распределенных системах и сервисах (Netflix, Uber, социальные сети) пользователи ожидают мгновенной реакции интерфейса на события, происходящие на сервере (завершение обработки видео, прибытие такси, новый лайк). Традиционный подход HTTP (Polling) неэффективен. Решением является связка:

Microservices -> Message Broker -> Notification Service -> WebSockets -> Client.

В микросервисной архитектуре не рекомендуется, чтобы каждый сервис открывал WebSocket-соединение с браузером. Это приведет к тому, что клиенту придется держать десятки открытых соединений. Вместо этого выделяется один **Notification Service**. Он:

1. **Слушает события от всех остальных сервисов через шину сообщений (RabbitMQ).**
2. **Держит единственное постоянное WebSocket-соединение с браузером пользователя.**
3. **Маршрутизирует сообщения из RabbitMQ в нужный WebSocket-канал.**

Технически сервис уведомлений выступает адаптером.

Вход:

Асинхронный Listener (@RabbitListener), принимающий DTO (например, UserRatedEvent).

Выход:

Синхронный вызов метода sendMessage у активной WebSocketSession.

Сложность:

Необходимо учитывать, что RabbitMQ работает в пуле потоков слушателей, а WebSocket-сессии хранятся в памяти. Операция записи в сокет должна быть потокобезопасной.

Практическая часть

Создание проекта Notification Service

Создайте новый Java Maven-проект notification-service.

Шаг 1. Настройка pom.xml Добавьте зависимости для Web, WebSockets, RabbitMQ и нашего контракта событий.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
```



| WebSockets

```
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.5.6</version>
    <relativePath/>
  </parent>
  <groupId>edu.rutmiit.demo</groupId>
  <artifactId>notification-service</artifactId>
  <version>0.0.1-SNAPSHOT</version>

  <properties>
    <java.version>17</java.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-
websocket</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-amqp</artifactId>
    </dependency>
    <dependency>
      <groupId>edu.rutmiit.demo</groupId>
      <artifactId>events-contract</artifactId>
      <version>1.2</version>
      <scope>system</scope>
      <systemPath>${project.basedir}/lib/events-
contract.jar</systemPath>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-
plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```



| WebSockets

Шаг 2. Настройка application.properties Сервис должен запускаться на уникальном порту.

```
spring.application.name=notification-service
server.port=8083
```

```
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest
```

Реализация WebSocket слоя

Реализуем логику управления соединениями, используя нативные веб-сокеты. Добавим пакет websocket, в нем создадим:

NotificationHandler

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;
import org.springframework.web.socket.CloseStatus;
import org.springframework.web.socket.TextMessage;
import org.springframework.web.socket.WebSocketSession;
import
org.springframework.web.socket.handler.TextWebSocketHandler;

import java.io.IOException;
import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;

@Component
public class NotificationHandler extends TextWebSocketHandler {

    private static final Logger log =
LoggerFactory.getLogger(NotificationHandler.class);

    // Потокбезопасный список сессий
    private final List<WebSocketSession> sessions = new
CopyOnWriteArrayList<>();

    @Override
    public void afterConnectionEstablished(WebSocketSession
session) {
        sessions.add(session);
        log.info("New WebSocket connection: {}",
session.getId());
    }
}
```



| WebSockets

```
    }

    @Override
    public void afterConnectionClosed(WebSocketSession session,
    CloseStatus status) {
        sessions.remove(session);
        log.info("WebSocket connection closed: {}",
    session.getId());
    }

    /**
     * Метод для отправки сообщения всем активным клиентам.
     * Вызывается из RabbitMQ Listener.
     */
    public void broadcast(String message) {
        for (WebSocketSession session : sessions) {
            if (session.isOpen()) {
                try {
                    synchronized (session) {
                        session.sendMessage(new
    TextMessage(message));
                    }
                } catch (IOException e) {
                    log.error("Error sending message to session
    {}", session.getId(), e);
                }
            }
        }
    }
}
```

WebSocketConfig

```
package edu.rutmiit.demo.websocket;

import org.springframework.context.annotation.Configuration;
import
org.springframework.web.socket.config.annotation.EnableWebSocket
;
import
org.springframework.web.socket.config.annotation.WebSocketConfig
urer;
import
org.springframework.web.socket.config.annotation.WebSocketHandle
rRegistry;

@Configuration
```



| WebSockets

```
@EnableWebSocket
public class WebSocketConfig implements WebSocketConfigurer {

    private final NotificationHandler notificationHandler;

    public WebSocketConfig(NotificationHandler
notificationHandler) {
        this.notificationHandler = notificationHandler;
    }

    @Override
    public void
registerWebSocketHandlers(WebSocketHandlerRegistry registry) {
        registry.addHandler(notificationHandler,
"/ws/notifications")
            .setAllowedOrigins("*"); // Для упрощения
лабораторной разрешаем все источники
    }
}
```

Реализация RabbitMQ слоя

Теперь создадим «мост», который будет слушать analytics-fanout и передавать данные в NotificationHandler. В новом пакете rabbitmq:

RabbitMQConfig

```
import
org.springframework.amqp.support.converter.Jackson2JsonMessageCo
nverter;
import
org.springframework.amqp.support.converter.MessageConverter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class RabbitMQConfig {
    @Bean
    public MessageConverter messageConverter() {
        return new Jackson2JsonMessageConverter();
    }
}
```

NotificationListener



| WebSockets

```
import edu.rutmiit.demo.events.UserRatedEvent;
import edu.rutmiit.demo.websocket.NotificationHandler;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.amqp.rabbit.annotation.Exchange;
import org.springframework.amqp.rabbit.annotation.Queue;
import org.springframework.amqp.rabbit.annotation.QueueBinding;
import
org.springframework.amqp.rabbit.annotation.RabbitListener;
import org.springframework.stereotype.Component;

@Component
public class NotificationListener {

    private static final Logger log =
LoggerFactory.getLogger(NotificationListener.class);
    private final NotificationHandler notificationHandler;

    public NotificationListener(NotificationHandler
notificationHandler) {
        this.notificationHandler = notificationHandler;
    }

    @RabbitListener(
        bindings = @QueueBinding(
            value = @Queue(name =
"q.notifications.browser", durable = "true"),
            exchange = @Exchange(name = "analytics-
fanout", type = "fanout")
        )
    )
    public void handleUserRatedEvent(UserRatedEvent event) {
        log.info("Received event from RabbitMQ: {}", event);

        // Формируем сообщение для пользователя
        String userMessage = String.format(
            "{ \"type\": \"RATING_UPDATE\", \"userId\": %d,
\"score\": %d, \"verdict\": \"%s\" }",
            event.userId(), event.score(), event.verdict()
        );

        // Отправляем в браузер
        notificationHandler.broadcast(userMessage);
    }
}
```

NotificationServiceApplication



```
package edu.rutmiit.demo;

import org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class Main {
    public static void main(String[] args) {
        SpringApplication.run(Main.class, args);
    }
}
```

Создание frontend-клиента

Создайте файл index.html в корне папки проекта (или в любом удобном месте).

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>Система уведомлений RUT-MIIT</title>
    <style>
        body { font-family: 'Segoe UI', Tahoma, Geneva,
Verdana, sans-serif; background-color: #f4f4f9;
padding: 20px; }
        .container { max-width: 600px; margin: 0 auto;
background: white; padding: 20px; border-radius: 8px;
box-shadow: 0 2px 10px rgba(0,0,0,0.1); }
        h2 { text-align: center; color: #333; }
        #status { text-align: center; margin-bottom:
20px; font-weight: bold; color: #dc3545; }
        .notification { padding: 15px; margin-bottom:
10px; border-left: 5px solid; border-radius: 4px;
animation: slideIn 0.5s ease-out; }
        .verdict-GOOD { background-color: #d4edda;
border-color: #28a745; color: #155724; }
        .verdict-BAD { background-color: #f8d7da;
border-color: #dc3545; color: #721c24; }
        @keyframes slideIn { from { transform:
translateX(100%); opacity: 0; } to { transform:
translateX(0); opacity: 1; } }
    </style>
</head>
<body>
<div class="container">
```



| WebSockets

```
<h2>Центр уведомлений</h2>
<div id="status">Отключено</div>
<div id="notifications-area"></div>
</div>

<script>
  const statusDiv =
document.getElementById('status');
  const area =
document.getElementById('notifications-area');
  let socket;

  function connect() {
    // Подключение к нашему новому сервису
    socket = new
WebSocket('ws://localhost:8083/ws/notifications');

    socket.onopen = () => {
      statusDiv.innerText = 'Подключено к системе
событий';
      statusDiv.style.color = '#28a745';
    };

    socket.onmessage = (event) => {
      const data = JSON.parse(event.data);
      console.log("Получено событие:", data);

      if (data.type === 'RATING_UPDATE') {
        showNotification(data);
      }
    };

    socket.onclose = () => {
      statusDiv.innerText = 'Соединение
разорвано. Переподключение...';
      statusDiv.style.color = '#dc3545';
      setTimeout(connect, 3000);
    };
  }

  function showNotification(data) {
    const div = document.createElement('div');
    div.className = `notification verdict-
```



| WebSockets

```
    ${data.verdict}`;  
    div.innerHTML = `  
        <strong>Пользователь ID:  
    ${data.userId}</strong><br>  
        Рейтинг рассчитан: <b>${data.score}</b><br>  
        Вердикт: ${data.verdict}  
    `;  
    area.prepend(div);  
}  
  
connect();  
</script>  
</body>  
</html>
```

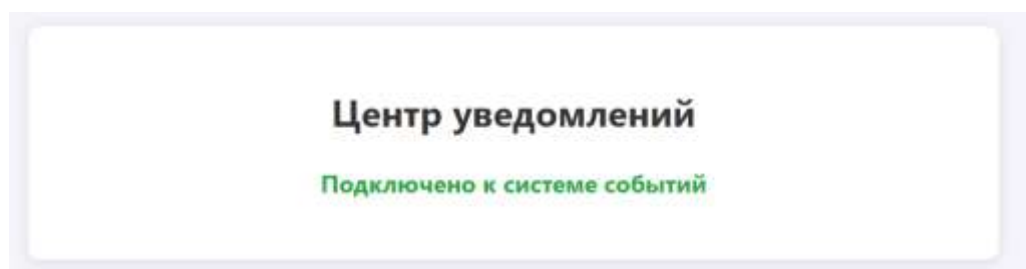
Запуск

Теперь необходимо запустить «оркестр» из 4-х компонентов.

1. RabbitMQ - убедитесь, что контейнер запущен.
2. Analytics Service - запустите `grpc.demo.Main` (Порт 9090).
3. Notification Service - запустите `NotificationServiceApplication`. (Порт 8083).
4. Demo REST - запустите `DemoRestApplication`. (Порт 8080).

Сценарий проверки:

1. Откройте файл `index.html` в браузере. Вы должны увидеть статус «Подключено к системе событий».



2. Откройте терминал или Postman.
3. Сделайте POST-запрос к `demo-rest`, чтобы инициировать расчет рейтинга.





WebSockets

Пользователь ID: 2
Рейтинг рассчитан: 60
Вердикт: GOOD

4. Наблюдайте:

1. demo-rest получит запрос.
2. Сходит в analytics-service по gRPC за рейтингом.
3. Отправит событие в RabbitMQ (analytics-fanout).
4. notification-service получит это событие.
5. NotificationListener передаст его в NotificationHandler.
6. WebSocket отправит JSON в браузер.
7. На странице мгновенно появится цветная карточка с рейтингом.